

Aim: To write a Python code to generate n-grams using NLTK n-gram Library

Description :

An **n-gram** is a **continuous sequence of n items** (typically words or characters) from a given text or speech. It's a fundamental concept in Natural Language Processing (NLP) and is used in many tasks, including **language modeling**, **text analysis**, **speech recognition**, and **machine translation**. **n** represents the number of items (usually words) in the sequence.

Types of N-Grams:

1. **Unigram (1-gram):**
 - A unigram is simply a single word (or character) from the text.
 - Example: "I love NLP"
 - Unigrams: ['I', 'love', 'NLP']
2. **Bigram (2-gram):**
 - A bigram is a sequence of two consecutive words.
 - Example: "I love NLP"
 - Bigrams: [('I', 'love'), ('love', 'NLP')]
3. **Trigram (3-gram):**
 - A trigram is a sequence of three consecutive words.
 - Example: "I love NLP"
 - Trigrams: [('I', 'love', 'NLP')]
4. **Tetragram (4-gram):**
 - A tetragram is a sequence of four consecutive words.
 - Example: "I love NLP very much"
 - Tetragrams: [('I', 'love', 'NLP', 'very'), ('love', 'NLP', 'very', 'much')]
5. **Higher-order n-grams (n > 3):**
 - You can generate n-grams for **any n**. For instance, you can create **5-grams**, **6-grams**, etc., depending on how much context you want to capture.

Use of N-Grams in NLP:

- **Language Modeling:** In speech recognition or text generation, n-grams are used to predict the likelihood of a word given its previous words. For example, in a bigram model, the probability of the word "*processing*" might depend on whether the previous word was "*language*".
 - **Example:**
 - Unigram Model: $P(\text{"language"}) = \frac{\text{count}(\text{"language"})}{\text{total words}}$
 - Bigram Model: $P(\text{"language"} | \text{"natural"}) = \frac{\text{count}(\text{"natural language"})}{\text{count}(\text{"natural"})}$
- **Text Classification:** N-grams can be used as features for tasks like sentiment analysis or spam detection.
- **Text Analysis:** N-grams help identify **common phrases** or **collocations** in the text.
- **Machine Translation:** N-grams are used to predict translations by matching **n-grams** in the source language with those in the target language.

Program :

```
import collections
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

def get_ngrams(text, n):
    """Generate n-grams from the given text."""
    tokens = word_tokenize(text.lower())
    n_grams = ngrams(tokens, n)
    return list(n_grams)

# Example text
text1 = "this is a sample text with several words this is another sample text with some different words"
text = "Sample list of words"

# Generate Uni-grams
print("List of Unigram")
ngrams_list = get_ngrams(text, 1)
for ngram in ngrams_list:
    print(ngram)

# Generate bi-grams
print("List of Bigrams")
ngrams_list = get_ngrams(text, 2)
for ngram in ngrams_list:
    print(ngram)

# Generate tri-grams
print("List of Trigrams")
ngrams_list = get_ngrams(text, 3)
for ngram in ngrams_list:
    print(ngram)
```

OUTPUT:

```
List of Unigram
('sample',)
('list',)
('of',)
('words',)
```

```
List of Bigrams
('sample', 'list')
('list', 'of')
('of', 'words')
```

```
List of Trigrams
('sample', 'list', 'of')
('list', 'of', 'words')
```